

개인 혈당데이터 분석용 RAG 기반 AI 상담 챗봇 소스코드

소스코드 (발체) — RAG 기반 AI 상담 챗봇

프로그램의 명칭	개인 혈당데이터 분석용 RAG 기반 AI 상담 챗봇 (Sejong AI Glucose Chatbot)
저작물의 종류	컴퓨터프로그램저작물
주요 기능	RAG(검색증강생성) 구조 의 혈당 상담 챗봇 — ①이용자 질의로 혈당 지식베이스를 검색(Retrieval)하여 일치 근거가 있으면 그 내용으로 응답하고 ②근거가 없으면 이용자의 혈당 분석 결과(TIR/TAR/TBR/CV 기반 유형 판정, 연령·당뇨 유형별 관리목표, 주의 음식·추천 운동)를 프롬프트에 결합(Augmentation)하여 ③생성형 AI가 상담 문장을 생성(Generation)한다 · 동일 질의 캐시 및 응답 진행 표시
개발 언어 및 환경	Java 17 (Spring Framework) · JavaScript · JSP · 생성형 AI API(Google Gemini generateContent) 연동
설계상의 특징	검색 단계는 혈당 지식베이스(자체 구축)에 대한 키워드 가중치 매칭 으로 구현하여 외부 벡터 DB 없이 동작한다. 수치 계산과 상태 판정은 프로그램이 직접 수행하고 생성형 AI에는 수치를 전달하지 않으며 정성적 표현만 결합해 문장 생성에만 사용한다. AI 미설정·호출 실패 시 자체 지표 요약으로 대체 응답한다.
본 문서의 범위	챗봇 모듈 소스 중 주요 기능 8개 부분 발체 (질의 처리·검색·증강·생성 각 단계와 지식베이스)

① 질의 접수 및 검색결과 증강 프롬프트 구성 — Augmentation (서버)

[src/main/java/egovframework/sejong/blood/web/BloodController.java]

```
988 @RequestMapping(value = "/blood/chatAsk.do", method = RequestMethod.POST)
989 @ResponseBody
990 public ResponseObject chatAsk(HttpSession session, @RequestBody HashMap<String, Object> params) {
991     ResponseObject json = new ResponseObject();
992
993     String q = params != null && params.get("q") != null ? params.get("q").toString().trim() : "";
994     if (q.isEmpty()) {
995         json.IsSucceed = false;
996         json.Message = "질문이 비어 있습니다.";
997         return json;
998     }
999     // 프롬프트 인젝션/과금 폭증 방지를 위한 길이 제한
1000     if (q.length() > 500) q = q.substring(0, 500);
1001
1002     if (geminiKey == null || geminiKey.isEmpty() || geminiKey.startsWith("YOUR_")
1003         || geminiUrl == null || geminiUrl.isEmpty()) {
1004         json.IsSucceed = false;
1005         (... 중략 ...)
1013 // == 프롬프트 - 기획 「AI 응답 Sample」(3차 전달자료_AI, 2026-08-13) 반영 ==
1014 // ★수치·유형 판정은 **화면(기존 자료)** 이 하고, LLM 은 **문장만** 만든다.
1015 //   ctx 에 TIR/TAR/TBR/CV 원값과 우리가 판정한 유형이 실려 온다(_chatCtxText).
1016 //   △LLM 에 판정을 맡기면 같은 수치에 다른 유형이 나온다 - 그래서 「다시 계산하지 말라」고 못박는다.
1017 //   ★답변 골격 = 기획안의 3단계 : ①현재 상태 → ②이유/피드백 → ③유지·개선 행동.
1018 //   원본 샘플이 그 구조일 때 "사용자가 즉시 이해하고 행동하기에 가장 효과적"이라고 결론냈다.
1019 String systemInstruction =
1020     "너는 당뇨/혈당 관리를 돕는 친절한 한국어 건강 도우미야. "
1021     + "사용자는 연속혈당측정(CGM)을 사용하는 환자야. "
1022     + "당변은 의학적 진단이 아니라 일반적인 생활관리 참고용이며, 심한 고혈당·저혈당이나 이상 증상은 반드시 담당 의사와
상담하라고 안내해. "
1023     + "당변은 100자 내외로 짧게, 줄바꿈은 <br> 태그로 표기해. "
1024     + "혈당 상태에 대한 질문이면 현재 상태 → 그렇게 보는 이유 → 지금 할 행동 순서로 각각 한 문장씩 써. "
1025     + "번호(㉠)나 머릿기호 없이 자연스러운 문장으로만 써. "
1026     + "아래 참고 정보는 앱이 이미 계산해 화면에 보여준 판정이야. 다시 계산하거나 바꾸지 말고 그대로 근거로 삼아. "
1027     + "숫자·퍼센트·지표 이름(TIR/TAR/TBR/CV)은 답변에 쓰지 마 - 수치는 사용자 화면 표가 이미 보여주고 있어. "
1028     + "판정유형별 방향 - 주수: 현재 식사·운동 유지를 격려 / 고혈당형: 식사량·탄수화물 비율 조절과 식후 걷기 / "
1029     + "저혈당형: 공복 운동 회피, 운동 전후 혈당 확인, 식사 거르지 않기 / 변동형: 식사·운동 시간을 일정하게. "
1030     (... 중략 ...)
1036 try {
1037     String answer = callGemini(systemInstruction, q);
1038     if (answer == null || answer.trim().isEmpty()) {
1039         json.IsSucceed = false;
```

① 질의 접수 및 검색결과 증강 프롬프트 구성 — Augmentation (서버) (이어서)

[src/main/java/egovframework/sejong/blood/web/BloodController.java]

```
1040         json.Message = "LLM 응답 없음";
1041         return json;
1042     }
1043     json.IsSucceed = true;
1044     json.Data = answer.trim();
1045     return json;
1046 } catch (Exception e) {
1047     log.error("chatAsk ERROR: " + e.getMessage(), e);
1048     json.IsSucceed = false;
1049     json.Message = "LLM 호출 오류";
1050     return json;
}
```

② 이용자 연령·당뇨 유형별 관리목표 결합 [src/main/java/egovframework/sejong/blood/web/BloodController.java]

```
1073     egovframework.sejong.util.CgmTarget t = egovframework.sejong.util.CgmTarget.of(age, type);
1074     StringBuilder who = new StringBuilder("이 사용자에게 적용되는 관리 목표는 ");
1075     who.append("TIR ").append(t.tir).append("% 이상, TAR ").append(t.tar).append("% 미만, TBR ").append(t.
tbr).append("% 미만")
1076         .append(" ").append(t.name).append(") 이야. 이 목표를 기준으로 말해. ");
1077     /* ★단정적인 말투를 쓰지 않게 - 이 앱은 진단을 하는 곳이 아니다. */
1078     who.append("상태를 단정하지 말고 '일 수 있어요', '~ 편이 좋습니다' 처럼 부드럽게 말해. ");
1079     return who.toString();
1080 } catch (Exception e) {
1081     log.warn("userProfileForPrompt 실패(무시하고 진행): {}", e.getMessage());
1082     return "";
1083 }
1084 }
```

③ 연령·당뇨 유형별 혈당관리 목표 기준 [src/main/java/egovframework/sejong/util/CgmTarget.java]

```
37 /** 나이·당뇨 유형에 맞는 기준. 값이 없으면(null) 일반 기준. */
38 public static CgmTarget of(Integer age, String typeName) {
39     if (typeName != null && typeName.startsWith("임신성")) {
40         return new CgmTarget("임신 중 기준", 70, 25, 4,
41             "임신 중에는 목표 범위가 63~140 mg/dL 로 더 좁습니다. 이 화면의 수치는 70~180 기준이라 담당 의사와 함께
확인해 주세요.");
42     }
43     if (age != null && age >= 65) {
44         return new CgmTarget("고령 기준", 50, 50, 1,
45             "나이를 고려해 저혈당을 더 엄격히, 고혈당은 조금 느슨하게 봅니다.");
46     }
47     return new CgmTarget("일반 기준", 70, 25, 4, "");
48 }
```

④ 생성형 AI(Gemini) 호출 — Generation 요청·응답 파싱

[src/main/java/egovframework/sejong/blood/web/BloodController.java]

```
1112 JSONObject genCfg = new JSONObject();
1113 genCfg.addProperty("temperature", 0.4);
1114 // 추론 토큰도 maxOutputTokens 를 함께 소모하므로 넉넉히 잡는다.
1115 // (1024 로 두면 추론에 900+ 를 써버려 답변이 잘림 = finishReason:MAX_TOKENS)
1116 genCfg.addProperty("maxOutputTokens", 2048);
1117 // thinking(추론) 최소화 - 단순 건강 Q&A 라 깊은 추론이 불필요.
1118 // ※ gemini-flash-latest 별칭이 Gemini 3 계열(gemini-3.6-flash)로 바뀌면서
1119 // 기존 thinkingBudget:0 은 400 INVALID_ARGUMENT 로 거부된다(추론 완전 비활성 불가).
1120 // Gemini 3 부터는 thinkingLevel("low"/"high") 을 사용한다.
1121 JSONObject thinkingCfg = new JSONObject();
1122 thinkingCfg.addProperty("thinkingLevel", "low");
1123 genCfg.add("thinkingConfig", thinkingCfg);
1124
1125 JSONObject reqBody = new JSONObject();
1126 reqBody.add("system_instruction", sysInstr);
1127 reqBody.add("contents", contents);
1128 reqBody.add("generationConfig", genCfg);
```

④ 생성형 AI(Gemini) 호출 — Generation 요청·응답 파싱 (이어서)

[src/main/java/egovframework/sejong/blood/web/BloodController.java]

```
1129
1130 // API 키는 헤더(x-goog-api-key)로 전달 (URL 쿼리 노출 방지)
1131 URL url = new URL(geminiUrl);
1132 HttpURLConnection conn = (HttpURLConnection) url.openConnection();
1133 conn.setRequestMethod("POST");
1134 conn.setRequestProperty("Content-Type", "application/json; charset=UTF-8");
1135 conn.setRequestProperty("x-goog-api-key", geminiKey);
    (... 중략 ...)
1157 // 응답 파싱: candidates[0].content.parts[*].text
1158 // Gemini 3 는 parts 에 추론 전용 조각(text 없이 thoughtSignature 만)이 섞여 올 수 있어
1159 // parts[0] 만 보면 null 이 되므로 text 가 있는 조각을 모두 이어붙인다.
1160 JsonObject root = gson.fromJson(sb.toString(), JsonObject.class);
1161 JsonArray candidates = root.getAsJsonArray("candidates");
1162 if (candidates == null || candidates.size() == 0) return null;
1163 JsonObject cand = candidates.get(0).getAsJsonObject();
1164 JsonObject candContent = cand.getAsJsonObject("content");
1165 if (candContent == null) return null;
1166 JsonArray parts = candContent.getAsJsonArray("parts");
1167 if (parts == null || parts.size() == 0) return null;
1168 StringBuilder text = new StringBuilder();
1169 for (int i = 0; i < parts.size(); i++) {
1170     JsonElement textEl = parts.get(i).getAsJsonObject().get("text");
1171     if (textEl != null && !textEl.isJsonNull()) text.append(textEl.getAsString());
1172 }
```

⑤질의 처리 흐름 — 검색 적중 시 즉답, 미적중 시 생성형 AI (화면) [src/main/webapp/WEB-INF/jsp/main/Blood_Consult.jsp]

```
1427 function sendChat(){
1428     if (_chatBusy) return; // [2026-08-05] 답변 대기 중 중복 전송 차단
1429     var input = document.getElementById('chatInput');
1430     var q = (input.value || '').trim(); if(!q) return;
1431     input.value = '';
1432     // 직접 입력으로 다른 질문을 하면 질문침 잠금 해제(같은 침을 다시 쓸 수 있게)
1433     if (q !== _lastQuickQ) _lastQuickQ = '';
1434     _addMsg(q, true);
1435
1436     // ㉠ 로컬 키워드/데이터 즉답 (blood_qa.js 매칭 + 화면 지표 기반)
1437     var local = _chatResponse(q);
1438     if (local != null) {
1439         _chatBusy = true;
1440         setTimeout(function(){
1441             _addMsg(local, false); _chatBusy = false;
1442             /* [2026-08-13] "내 최고혈당 관련 운동, 식사 가이드"처럼 수치+가이드를 함께 물으면
1443             수치 즉답으로 끝내지 않고 LLM 가이드 답변을 이어붙인다(사용자 지적).
1444             화면의 주의음식·추천운동 TOP3 도 참고자료로 함께 넣는다. */
1445             if (_chatLLMFollow) { _chatLLMFollow = false; _askLLM(q, _chatGuideCtx()); }
1446         }, 280);
1447     }
1448     return;
```

⑥ AI 참고자료 생성 및 혈당 유형 4분류 자동 판정 [src/main/webapp/WEB-INF/jsp/main/Blood_Consult.jsp]

```
1783 function _chatCtxText(){
1784     var tir = _metricNum('tir'), tar = _metricNum('tar'), tbr = _metricNum('tbr'), cv = _metricNum('cv');
1785     if (tir == null && tar == null && tbr == null && cv == null) return '';
1786     var parts = [];
1787     if (tir != null) parts.push(tir >= 70 ? '목표범위 유지 양호' : (tir >= 50 ? '목표범위 유지 다소 부족' : '목표범
1788     if (tar != null) parts.push(tar >= 25 ? '고혈당 시간 김' : (tar >= 10 ? '고혈당 시간 다소 있음' : '고혈당
1789     if (tbr != null) parts.push(tbr >= 4 ? '저혈당 반복' : (tbr >= 1 ? '저혈당 가끔' : '저혈
1790     if (cv != null) parts.push(cv > 36 ? '혈당 변동 큼' : '혈당 변동 안정적');
1791     var t = _chatGlucoseType(tir, tar, tbr, cv);
1792     return parts.join(' / ') + (t ? (' / 판정유형 ' + t) : '');
1793 }
```

⑥ AI 참고자료 생성 및 혈당 유형 4분류 자동 판정 (이어서) [src/main/webapp/WEB-INF/jsp/main/Blood_Consult.jsp]

```
1793 }
1794
1795 /* 혈당 유형 자동 판정 - 기획안 §「혈당 유형 자동 판정」 4유형.
1796 ★판정은 **우리가 한다**(대한당뇨병학회 CGM 기준: TIR≥70·TAR<25·TBR<4·CV≤36).
1797 LLM 에 맡기면 같은 수치에 다른 유형이 나올 수 있다 - 수치·판정은 기존 자료, 문장만 AI. */
1798 function _chatGlucoseType(tir, tar, tbr, cv){
1799     if (tir == null && tar == null && tbr == null) return '';
1800     if (tbr != null && tbr >= 4) return '저혈당형';
1801     if (tar != null && tar >= 25) return '고혈당형';
1802     if (cv != null && cv > 36) return '변동형';
1803     if (tir != null && tir >= 70) return '우수';
1804     return '변동형';
1805 }
```

⑦ 지식베이스 검색 — Retrieval (키워드 가중치 점수제) [src/main/webapp/WEB-INF/jsp/main/Blood_Consult.jsp]

```
1729 var bestItem = null, bestScore = 0, bestMaxLen = 0, secondScore = 0;
1730 for (var i = 0; i < BLOOD_QA.length; i++) {
1731     var item = BLOOD_QA[i];
1732     if (!item || !item.kw) continue;
1733     var score = 0, maxLen = 0;
1734     for (var j = 0; j < item.kw.length; j++) {
1735         var kw = String(item.kw[j]).toLowerCase();
1736         if (kw && q.indexOf(kw) !== -1) {
1737             score += kw.length;
1738             if (kw.length > maxLen) maxLen = kw.length;
1739         }
1740     }
1741     if (score > bestScore) { secondScore = bestScore; bestItem = item; bestScore = score; bestMaxLen =
maxLen; }
1742     else if (score > secondScore) { secondScore = score; }
1743 }
1744 if (bestItem && bestMaxLen >= 2 && bestScore >= secondScore + 2) {
1745     return bestItem.a;
1746 }
1747 }
```

⑧ 혈당 지식베이스(Knowledge Base) 자료 구조 [src/main/webapp/asset/js/blood_qa.js]

```
1 /* =====
2   혈당 Q&A 지식베이스 (데이터)
3   -----
4   · 새 질문/답변을 추가하려면 아래 BLOOD_QA 배열에 항목만 추가하세요.
5     (소스 로직 수정 불필요 - 이 파일만 편집)
6   · 형식: { kw: ['키워드1', '키워드2', ...], a: '답변 HTML' }
7     - kw : 질문에 이 키워드 중 하나라도 포함되면 매칭 (대소문자 무시)
8     - a  : 표시할 답변 (HTML 허용, 줄바꿈은 <br>)
9   · 위에 있는 항목이 먼저 매칭됩니다. 더 구체적인 항목을 위쪽에 두세요.
10  · 영문 키워드는 소문자로 작성하세요 (예: 'hba1c').
11  · "오늘 평균/최고/최저/추이" 같은 데이터 계산형 답변은 코드(_chatResponse)
12    에서 처리하므로 여기에 넣지 않습니다.
13  ===== */
14 var BLOOD_QA = [
```

※ 본 문서는 챗봇 모듈 소스코드 중 주요 부분을 발췌한 것이며, 각 코드 왼쪽의 번호는 해당 파일 내 실제 줄번호이다. 가독성을 위해 발췌 구간의 공통 들여쓰기만 제거하였고 코드 내용은 원본과 동일하다. 인증키 등 비밀값은 소스에 포함되지 않고 실행 환경의 환경변수로 주입된다.